

Bank System

Generated by Doxygen 1.9.8

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 sClient Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 AccountBalance	5
3.1.2.2 AccountNumber	5
3.1.2.3 MarkForDelete	6
3.1.2.4 Name	6
3.1.2.5 Phone	6
3.1.2.6 PinCode	6
4 File Documentation	7
4.1 Bank_System.cpp File Reference	7
4.1.1 Enumeration Type Documentation	9
4.1.1.1 enMainMenuOptions	9
4.1.1.2 enTransactionsMenuOptions	10
4.1.2 Function Documentation	10
4.1.2.1 AddNewClient()	10
4.1.2.2 AddNewClients()	11
4.1.2.3 ChangeClientRecord()	12
4.1.2.4 ClientExistsByAccountNumber()	13
4.1.2.5 ConvertLinetoRecord()	14
4.1.2.6 ConvertRecordToLine()	15
4.1.2.7 DeleteClientByAccountNumber()	16
4.1.2.8 DepositBalanceToClientByAccountNumber()	17
4.1.2.9 FindClientByAccountNumber()	18
4.1.2.10 GoBackToMainMenu()	19
4.1.2.11 GoBackToTransactionsMenu()	20
4.1.2.12 LoadCleintsDataFromFile()	21
4.1.2.13 main()	22
4.1.2.14 MarkClientForDeleteByAccountNumber()	23
4.1.2.15 PerfromMainMenuOption()	23
4.1.2.16 PerfromTranactionsMenuOption()	25
4.1.2.17 PrintClientCard()	26
4.1.2.18 PrintClientRecordBalanceLine()	27
4.1.2.19 PrintClientRecordLine()	28
4.1.2.20 ReadClientAccountNumber()	28

4.1.2.21 ReadMainMenueOption()	29
4.1.2.22 ReadNewClient()	30
4.1.2.23 ReadTransactionsMenueOption()	31
4.1.2.24 SaveCleintsDataToFile()	31
4.1.2.25 ShowAddNewClientsScreen()	32
4.1.2.26 ShowAllClientsScreen()	33
4.1.2.27 ShowDeleteClientScreen()	34
4.1.2.28 ShowDepositScreen()	35
4.1.2.29 ShowEndScreen()	36
4.1.2.30 ShowFindClientScreen()	37
4.1.2.31 ShowMainMenue()	38
4.1.2.32 ShowTotalBalances()	39
4.1.2.33 ShowTotalBalancesScreen()	40
4.1.2.34 ShowTransactionsMenue()	40
4.1.2.35 ShowUpdateClientScreen()	41
4.1.2.36 ShowWithdrawScreen()	42
4.1.2.37 SplitString()	43
4.1.2.38 UpdateClientByAccountNumber()	44
4.1.3 Variable Documentation	45
4.1.3.1 ClientsFileName	45
4.2 Bank_System.cpp	46

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

sClient	5
-----------------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Bank_System.cpp		7
---------------------------------	-------	---

Chapter 3

Data Structure Documentation

3.1 sClient Struct Reference

Data Fields

- string [AccountNumber](#)
- string [PinCode](#)
- string [Name](#)
- string [Phone](#)
- double [AccountBalance](#)
- bool [MarkForDelete](#) = false

3.1.1 Detailed Description

Definition at line 13 of file [Bank_System.cpp](#).

3.1.2 Field Documentation

3.1.2.1 AccountBalance

```
double sClient::AccountBalance
```

Definition at line 19 of file [Bank_System.cpp](#).

Referenced by [ChangeClientRecord\(\)](#), [ConvertLinetoRecord\(\)](#), [ConvertRecordToLine\(\)](#), [PrintClientCard\(\)](#), [PrintClientRecordBalanceLine\(\)](#), [PrintClientRecordLine\(\)](#), [ReadNewClient\(\)](#), and [ShowWithdrawScreen\(\)](#).

3.1.2.2 AccountNumber

```
string sClient::AccountNumber
```

Definition at line 15 of file [Bank_System.cpp](#).

Referenced by [ChangeClientRecord\(\)](#), [ClientExistsByAccountNumber\(\)](#), [ConvertLinetoRecord\(\)](#), [ConvertRecordToLine\(\)](#), [PrintClientCard\(\)](#), [PrintClientRecordBalanceLine\(\)](#), [PrintClientRecordLine\(\)](#), and [ReadNewClient\(\)](#).

3.1.2.3 MarkForDelete

```
bool sClient::MarkForDelete = false
```

Definition at line 20 of file [Bank_System.cpp](#).

Referenced by [MarkClientForDeleteByAccountNumber\(\)](#).

3.1.2.4 Name

```
string sClient::Name
```

Definition at line 17 of file [Bank_System.cpp](#).

Referenced by [ChangeClientRecord\(\)](#), [ConvertLinetoRecord\(\)](#), [ConvertRecordToLine\(\)](#), [PrintClientCard\(\)](#), [PrintClientRecordBalanceLine\(\)](#), [PrintClientRecordLine\(\)](#), and [ReadNewClient\(\)](#).

3.1.2.5 Phone

```
string sClient::Phone
```

Definition at line 18 of file [Bank_System.cpp](#).

Referenced by [ChangeClientRecord\(\)](#), [ConvertLinetoRecord\(\)](#), [ConvertRecordToLine\(\)](#), [PrintClientCard\(\)](#), [PrintClientRecordLine\(\)](#), and [ReadNewClient\(\)](#).

3.1.2.6 PinCode

```
string sClient::PinCode
```

Definition at line 16 of file [Bank_System.cpp](#).

Referenced by [ChangeClientRecord\(\)](#), [ConvertLinetoRecord\(\)](#), [ConvertRecordToLine\(\)](#), [PrintClientCard\(\)](#), [PrintClientRecordLine\(\)](#), and [ReadNewClient\(\)](#).

The documentation for this struct was generated from the following file:

- [Bank_System.cpp](#)

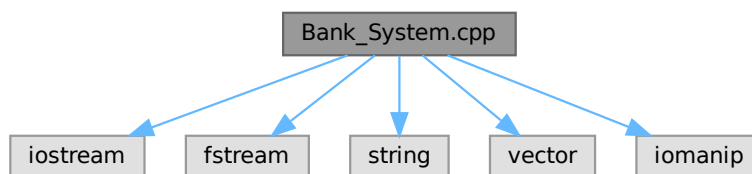
Chapter 4

File Documentation

4.1 Bank_System.cpp File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>
```

Include dependency graph for Bank_System.cpp:



Data Structures

- struct [sClient](#)

Enumerations

- enum [enTransactionsMenuOptions](#) { [eDeposit](#) = 1 , [eWithdraw](#) = 2 , [eShowTotalBalance](#) = 3 , [eShowMainMenu](#) = 4 }
- enum [enMainMenuOptions](#) { [eListClients](#) = 1 , [eAddNewClient](#) = 2 , [eDeleteClient](#) = 3 , [eUpdateClient](#) = 4 , [eFindClient](#) = 5 , [eShowTransactionsMenu](#) = 6 , [eExit](#) = 7 }

Functions

- void [ShowMainMenu](#) ()
すべてのクライアントの画面を表示する
- void [ShowTransactionsMenu](#) ()
すべてのクライアントの画面を表示する
- vector< string > [SplitString](#) (string S1, string Delim)
文字列を指定した区切り文字で分割する
- [sClient](#) [ConvertLinetoRecord](#) (string Line, string Seperator="#//#")
文字列をレコードに変換する関数
- string [ConvertRecordToLine](#) ([sClient](#) Client, string Seperator="#//#")
クライアントのレコードを一行の文字列に変換する
- bool [ClientExistsByAccountNumber](#) (string AccountNumber, string FileName)
アカウント番号に基づいてクライアントが存在するかどうかを確認する関数
- [sClient](#) [ReadNewClient](#) ()
新規クライアントを読み込む
- vector< [sClient](#) > [LoadCleintsDataFromFile](#) (string FileName)
ファイルからクライアントのデータを読み込み、それを *sClient* 型のベクタとして返す関数
- void [PrintClientRecordLine](#) ([sClient](#) Client)
クライアントのレコードを一行で出力する
- void [PrintClientRecordBalanceLine](#) ([sClient](#) Client)
クライアントのレコードを一行で出力する
- void [ShowAllClientsScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowTotalBalances](#) ()
すべてのクライアントの画面を表示する
- void [PrintClientCard](#) ([sClient](#) Client)
クライアントのレコードを一行で出力する
- bool [FindClientByAccountNumber](#) (string AccountNumber, vector< [sClient](#) > vClients, [sClient](#) &Client)
アカウント番号を用いてクライアントを検索する関数
- [sClient](#) [ChangeClientRecord](#) (string AccountNumber)
与えられたアカウント番号に基づいてクライアントのレコードを変更する
- bool [MarkClientForDeleteByAccountNumber](#) (string AccountNumber, vector< [sClient](#) > &vClients)
アカウント番号によってクライアントを削除マークします。
- vector< [sClient](#) > [SaveCleintsDataToFile](#) (string FileName, vector< [sClient](#) > vClients)
クライアントデータを指定のファイルに保存する関数
- void [AddNewClient](#) ()
ファイルにデータ行を追加する
- void [AddNewClients](#) ()
すべてのクライアントの画面を表示する
- bool [DeleteClientByAccountNumber](#) (string AccountNumber, vector< [sClient](#) > &vClients)
アカウント番号によってクライアントを削除マークします。
- bool [UpdateClientByAccountNumber](#) (string AccountNumber, vector< [sClient](#) > &vClients)
アカウント番号によってクライアントを削除マークします。
- bool [DepositBalanceToClientByAccountNumber](#) (string AccountNumber, double Amount, vector< [sClient](#) > &vClients)
指定された口座番号のクライアントに対して、指定された金額を預金します。
- string [ReadClientAccountNumber](#) ()
クライアントのアカウント番号を読み取る
- void [ShowDeleteClientScreen](#) ()
すべてのクライアントの画面を表示する

- void [ShowUpdateClientScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowAddNewClientsScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowFindClientScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowEndScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowDepositScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowWithdrawScreen](#) ()
すべてのクライアントの画面を表示する
- void [ShowTotalBalancesScreen](#) ()
すべてのクライアントの画面を表示する
- void [GoBackToMainMenu](#) ()
すべてのクライアントの画面を表示する
- void [GoBackToTransactionsMenu](#) ()
すべてのクライアントの画面を表示する
- short [ReadTransactionsMenuOption](#) ()
トランザクションメニューオプションを読み込む関数
- void [PerfromTranactionsMenuOption](#) ([enTransactionsMenuOptions](#) TransactionMenuOption)
トランザクションメニューオプションを実行する関数
- short [ReadMainMenuOption](#) ()
トランザクションメニューオプションを読み込む関数
- void [PerfromMainMenuOption](#) ([enMainMenuOptions](#) MainMenuOption)
メインメニューオプションを実行する関数
- int [main](#) ()
メイン関数

Variables

- const string [ClientsFileName](#) = "Clients.txt"

4.1.1 Enumeration Type Documentation

4.1.1.1 enMainMenuOptions

enum [enMainMenuOptions](#)

Enumerator

eListClients	
eAddNewClient	
eDeleteClient	
eUpdateClient	
eFindClient	
eShowTransactionsMenu	
eExit	

Definition at line 750 of file [Bank_System.cpp](#).

```
00751 {
00752     eListClients = 1,
00753     eAddNewClient = 2,
00754     eDeleteClient = 3,
00755     eUpdateClient = 4,
00756     eFindClient = 5,
00757     eShowTransactionsMenue = 6,
00758     eExit = 7
00759 };
```

4.1.1.2 enTransactionsMenueOptions

enum [enTransactionsMenueOptions](#)

Enumerator

eDeposit	
eWithdraw	
eShowTotalBalance	
eShowMainMenue	

Definition at line 742 of file [Bank_System.cpp](#).

```
00743 {
00744     eDeposit = 1,
00745     eWithdraw = 2,
00746     eShowTotalBalance = 3,
00747     eShowMainMenue = 4
00748 };
```

4.1.2 Function Documentation

4.1.2.1 AddNewClient()

```
void AddNewClient ( )
```

ファイルにデータ行を追加する

Parameters

<i>FileName</i>	(string) 追加先のファイル名
<i>stDataLine</i>	(string) 追加するデータ行

この関数は指定されたファイル名のファイルに、指定されたデータ行を追加します。データ行は新たな行として追加されます。ファイルが存在しない場合は新規に作成されます。void AddDataLineToFile(string FileName, string stDataLine) { fstream MyFile; MyFile.open(FileName, ios::out | ios::app);

```
if (MyFile.is_open()) {
```

```
MyFile << stDataLine << endl;
```

```
MyFile.close(); } }
```

```
/** すべてのクライアントの画面を表示する
```

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 447 of file [Bank_System.cpp](#).

```
00448 {
00449     sClient Client;
00450     Client = ReadNewClient();
00451     AddDataLineToFile(ClientsFileName, ConvertRecordToLine(Client));
00452 }
```

References [ClientsFileName](#), [ConvertRecordToLine\(\)](#), and [ReadNewClient\(\)](#).

Referenced by [AddNewClients\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.2 AddNewClients()

```
void AddNewClients ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 457 of file [Bank_System.cpp](#).

```
00458 {
00459     char AddMore = 'Y';
00460     do
00461     {
00462         // system("cls");
00463         cout << "Adding New Client:\n\n";
00464
00465         AddNewClient();
00466         cout << "\nClient Added Successfully, do you want to add more clients? Y/N? ";
00467
00468         cin >> AddMore;
00469     } while (toupper(AddMore) == 'Y');
00470 }
00471 }
```

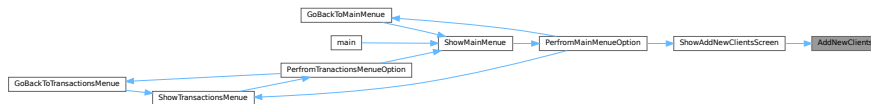
References [AddNewClient\(\)](#).

Referenced by [ShowAddNewClientsScreen\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.3 ChangeClientRecord()

```
sClient ChangeClientRecord (
    string AccountNumber )
```

与えられたアカウント番号に基づいてクライアントのレコードを変更する

Parameters

<i>AccountNumber</i>	(string) 変更を行いたいクライアントのアカウント番号
----------------------	--------------------------------

Returns

sClient 変更後のクライアント情報を含む sClient オブジェクト

この関数は、指定されたアカウント番号を持つクライアントのレコードを変更します。変更後のクライアント情報は sClient オブジェクトとして返されます。

Definition at line 349 of file [Bank_System.cpp](#).

```

00350 {
00351     sClient Client;
00352
00353     Client.AccountNumber = AccountNumber;
00354
00355     cout << "\n\nEnter PinCode? ";
00356     getline(cin >> ws, Client.PinCode);
00357
00358     cout << "Enter Name? ";
00359     getline(cin, Client.Name);
00360
00361     cout << "Enter Phone? ";
00362     getline(cin, Client.Phone);
00363
00364     cout << "Enter AccountBalance? ";
00365     cin >> Client.AccountBalance;
00366
00367     return Client;
00368 }
```

References [sClient::AccountBalance](#), [sClient::AccountNumber](#), [sClient::Name](#), [sClient::Phone](#), and [sClient::PinCode](#).

Referenced by [UpdateClientByAccountNumber\(\)](#).

Here is the caller graph for this function:



4.1.2.4 ClientExistsByAccountNumber()

```
bool ClientExistsByAccountNumber (
    string AccountNumber,
    string FileName )
```

アカウント番号に基づいてクライアントが存在するかどうかを確認する関数

Parameters

<i>AccountNumber</i>	(string) 確認したいクライアントのアカウント番号
<i>FileName</i>	(string) クライアント情報が格納されているファイルの名前

Returns

bool クライアントが存在する場合は true、存在しない場合は false を返す

ファイルからクライアント情報を読み込み、指定されたアカウント番号のクライアントが存在するかどうかを確認します。

Definition at line 104 of file [Bank_System.cpp](#).

```

00105 {
00106
00107     vector<sClient> vClients;
00108
00109     fstream MyFile;
00110     MyFile.open(FileName, ios::in); // read Mode
00111
00112     if (MyFile.is_open())
00113     {
00114
00115         string Line;
00116         sClient Client;
00117
00118         while (getline(MyFile, Line))
00119         {
00120
00121             Client = ConvertLinetoRecord(Line);
00122             if (Client.AccountNumber == AccountNumber)
00123             {
00124                 MyFile.close();
00125                 return true;
00126             }
00127
00128             vClients.push_back(Client);
00129         }
00130
00131         MyFile.close();
00132     }
00133
00134     return false;
00135 }
```

References [sClient::AccountNumber](#), and [ConvertLinetoRecord\(\)](#).

Referenced by [ReadNewClient\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.5 ConvertLinetoRecord()

```

sClient ConvertLinetoRecord (
    string Line,
    string Seperator = "#/#/" )
  
```

文字列をレコードに変換する関数

Parameters

<i>Line</i>	(string) レコードに変換するための文字列
<i>Seperator</i>	(string) フィールドを区切るためのセパレータ（デフォルトは "#/#/"）

Returns

[sClient](#) 変換されたレコード

この関数は、指定されたセパレータを使用して文字列をフィールドに分割し、その結果を [sClient](#) 型のレコードに変換します。セパレータが指定されていない場合はデフォルトの "#/#/" を使用します。

Definition at line 62 of file [Bank_System.cpp](#).

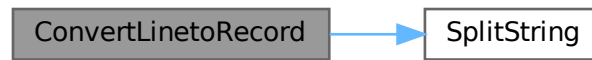
```

00063 {
00064     sClient Client;
00065     vector<string> vClientData;
00066
00067     vClientData = SplitString(Line, Seperator);
00068
00069     Client.AccountNumber = vClientData[0];
00070     Client.PinCode = vClientData[1];
00071     Client.Name = vClientData[2];
00072     Client.Phone = vClientData[3];
00073     Client.AccountBalance = stod(vClientData[4]); // cast string to double
00074
00075     return Client;
00076 }
  
```

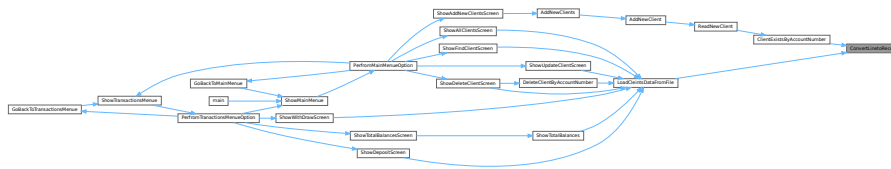
References [sClient::AccountBalance](#), [sClient::AccountNumber](#), [sClient::Name](#), [sClient::Phone](#), [sClient::PinCode](#), and [SplitString\(\)](#).

Referenced by [ClientExistsByAccountNumber\(\)](#), and [LoadCleintsDataFromFile\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.6 ConvertRecordToLine()

```
string ConvertRecordToLine (
    sClient Client,
    string Seperator = "#//#" )
```

クライアントのレコードを一行の文字列に変換する

Parameters

<i>Client</i>	(sClient) 変換されるクライアントのレコード
<i>Seperator</i>	(string) レコードの各要素を分割するためのセパレータ。デフォルトは "#/#/#"

Returns

string 変換された一行の文字列

この関数は、クライアントのレコードを一行の文字列に変換します。各要素は指定されたセパレータで分割されます。セパレータが指定されない場合、デフォルトのセパレータ `"/#/` が使用されます。

Definition at line 84 of file Bank_System.cpp.

```

00085 {
00086
00087     string stClientRecord = "";
00088
00089     stClientRecord += Client.AccountNumber + Seperator;
00090     stClientRecord += Client.PinCode + Seperator;
00091     stClientRecord += Client.Name + Seperator;
00092     stClientRecord += Client.Phone + Seperator;
00093     stClientRecord += to_string(Client.AccountBalance);
00094

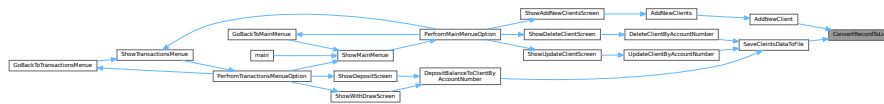
```

```
00095     return stClientRecord;
00096 }
```

References [sClient::AccountBalance](#), [sClient::AccountNumber](#), [sClient::Name](#), [sClient::Phone](#), and [sClient::PinCode](#).

Referenced by [AddNewClient\(\)](#), and [SaveCleintsDataToFile\(\)](#).

Here is the caller graph for this function:



4.1.2.7 DeleteClientByAccountNumber()

```
bool DeleteClientByAccountNumber (
    string AccountNumber,
    vector< sClient > & vClients )
```

アカウント番号によってクライアントを削除マークします。

Parameters

<i>AccountNumber</i>	(string) 削除マークをつけるクライアントのアカウント番号
<i>vClients</i>	(vector< sClient > &) クライアント情報が格納されたベクター

Returns

bool 削除マークが正常につけられた場合は true、それ以外の場合は false を返します。

この関数は指定されたアカウント番号のクライアントを探し、見つかった場合にそのクライアントに削除マークをつけます。クライアントが見つからなかった場合や何らかのエラーが発生した場合は false を返します。

Definition at line 479 of file [Bank_System.cpp](#).

```
00480 {
00481
00482     sClient Client;
00483     char Answer = 'n';
00484
00485     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00486     {
00487
00488         PrintClientCard(Client);
00489
00490         cout << "\n\nAre you sure you want delete this client? y/n ? ";
00491         cin >> Answer;
00492         if (Answer == 'y' || Answer == 'Y')
00493         {
00494             MarkClientForDeleteByAccountNumber(AccountNumber, vClients);
00495             SaveCleintsDataToFile(ClientsFileName, vClients);
00496
00497             // Refresh Clients
00498             vClients = LoadCleintsDataFromFile(ClientsFileName);
00499
00500             cout << "\n\nClient Deleted Successfully.";
00501             return true;
00502         }
00503     }
```

```

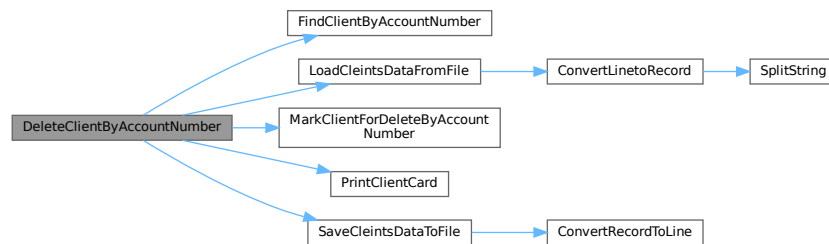
00504     else
00505     {
00506         cout << "\nClient with Account Number (" << AccountNumber << ") is Not Found!";
00507         return false;
00508     }
00509 }

```

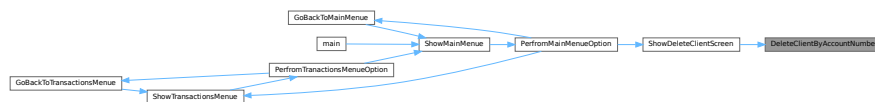
References [ClientsFileName](#), [FindClientByAccountNumber\(\)](#), [LoadCleintsDataFromFile\(\)](#), [MarkClientForDeleteByAccountNumber\(\)](#), [PrintClientCard\(\)](#), and [SaveCleintsDataToFile\(\)](#).

Referenced by [ShowDeleteClientScreen\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.8 DepositBalanceToClientByAccountNumber()

```

bool DepositBalanceToClientByAccountNumber (
    string AccountNumber,
    double Amount,
    vector< sClient > & vClients )

```

指定された口座番号のクライアントに対して、指定された金額を預金します。

Parameters

<i>AccountNumber</i>	(string) 預金を行うクライアントの口座番号。
<i>Amount</i>	(double) 預金する金額。
<i>vClients</i>	(vector< sClient > &) クライアント情報を保持するベクター。

Returns

bool 預金が成功した場合は true、それ以外の場合は false を返します。

この関数は、指定された口座番号のクライアントに対して指定された金額を預金します。預金を行うクライアントは `vClients` ベクターから検索されます。口座番号が存在しない場合や預金処理が何らかの理由で失敗した場合は `false` を返します。成功した場合は `true` を返します。

Definition at line 561 of file [Bank_System.cpp](#).

```
00562 {
00563     char Answer = 'n';
00564
00565     cout << "\n\nAre you sure you want perfrom this transaction? y/n ? ";
00566     cin >> Answer;
00567     if (Answer == 'y' || Answer == 'Y')
00568     {
00569
00570         for (sClient &C : vClients)
00571         {
00572             if (C.AccountNumber == AccountNumber)
00573             {
00574                 C.AccountBalance += Amount;
00575                 SaveCleintsDataToFile(ClientsFileName, vClients);
00576                 cout << "\n\nDone Successfully. New balance is: " << C.AccountBalance;
00577
00578                 return true;
00579             }
00580         }
00581         return false;
00582     }
00583 }
```

References [ClientsFileName](#), and [SaveCleintsDataToFile\(\)](#).

Referenced by [ShowDepositScreen\(\)](#), and [ShowWithdrawScreen\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.9 FindClientByAccountNumber()

```
bool FindClientByAccountNumber (
    string AccountNumber,
    vector< sClient > vClients,
    sClient & Client )
```

アカウント番号を用いてクライアントを検索する関数

Parameters

<i>AccountNumber</i>	(string) 検索するクライアントのアカウント番号
<i>vClients</i>	(vector< sClient >) クライアント情報が格納されたベクター
<i>Client</i>	(sClient &) 検索結果を格納するためのクライアントオブジェクトの参照

Returns

bool 検索結果。クライアントが見つかった場合は true、見つからなかった場合は false を返す。

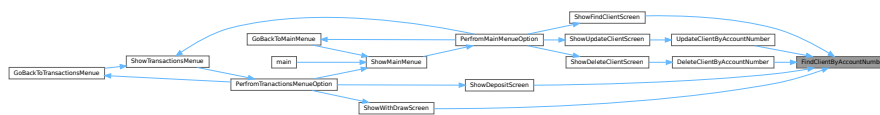
アカウント番号を用いてクライアント情報を検索します。見つかった場合は該当するクライアント情報を Client オブジェクトに格納し、true を返します。見つからなかった場合は false を返します。

Definition at line 329 of file [Bank_System.cpp](#).

```
00330 {
00331
00332     for (sClient C : vClients)
00333     {
00334
00335         if (C.AccountNumber == AccountNumber)
00336         {
00337             Client = C;
00338             return true;
00339         }
00340     }
00341     return false;
00342 }
```

Referenced by [DeleteClientByAccountNumber\(\)](#), [ShowDepositScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowWithdrawScreen\(\)](#), and [UpdateClientByAccountNumber\(\)](#).

Here is the caller graph for this function:



4.1.2.10 GoBackToMainMenu()

```
void GoBackToMainMenu ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 764 of file [Bank_System.cpp](#).

```
00765 {
00766     cout << "\n\nPress any key to go back to Main Menue...";
00767     system("pause>0");
00768     ShowMainMenu();
00769 }
```

References [ShowMainMenu\(\)](#).

Referenced by [PerfromMainMenuOption\(\)](#).

Here is the caller graph for this function:



4.1.2.12 LoadCleintsDataFromFile()

```
vector< sClient > LoadCleintsDataFromFile (
    string FileName )
```

ファイルからクライアントのデータを読み込み、それを sClient 型のベクタとして返す関数

Parameters

FileName	(string) クライアントのデータが保存されているファイルの名前
-----------------	------------------------------------

Returns

vector< sClient > ファイルから読み込んだクライアントのデータを格納した sClient 型のベクタ

この関数は指定されたファイルからクライアントのデータを読み込みます。読み込んだデータは sClient 型のベクタとして返されます。ファイルが存在しない場合や読み込みに失敗した場合のエラーハンドリングはこの関数の内部で行われます。

Definition at line 176 of file [Bank_System.cpp](#).

```

00177 {
00178
00179     vector<sClient> vClients;
00180
00181     fstream MyFile;
00182     MyFile.open(FileName, ios::in); // read Mode
00183
00184     if (MyFile.is_open())
00185     {
00186
00187         string Line;
00188         sClient Client;
00189
00190         while (getline(MyFile, Line))
00191         {
00192
00193             Client = ConvertLinetoRecord(Line);
00194
00195             vClients.push_back(Client);
00196         }
00197
00198         MyFile.close();
00199     }
00200
00201     return vClients;
00202 }
```

References [ConvertLinetoRecord\(\)](#).

Referenced by [DeleteClientByAccountNumber\(\)](#), [ShowAllClientsScreen\(\)](#), [ShowDeleteClientScreen\(\)](#), [ShowDepositScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowTotalBalances\(\)](#), [ShowUpdateClientScreen\(\)](#), and [ShowWithdrawScreen\(\)](#).

4.1.2.14 MarkClientForDeleteByAccountNumber()

```
bool MarkClientForDeleteByAccountNumber (
    string AccountNumber,
    vector< sClient > & vClients )
```

アカウント番号によってクライアントを削除マークします。

Parameters

<i>AccountNumber</i>	(string) 削除マークをつけるクライアントのアカウント番号
<i>vClients</i>	(vector< sClient > &) クライアント情報が格納されたベクター

Returns

bool 削除マークが正常につけられた場合は true、それ以外の場合は false を返します。

この関数は指定されたアカウント番号のクライアントを探し、見つかった場合にそのクライアントに削除マークをつけます。クライアントが見つからなかった場合や何らかのエラーが発生した場合は false を返します。

Definition at line 376 of file [Bank_System.cpp](#).

```
00377 {
00378
00379     for (sClient &C : vClients)
00380     {
00381
00382         if (C.AccountNumber == AccountNumber)
00383         {
00384             C.MarkForDelete = true;
00385             return true;
00386         }
00387     }
00388
00389     return false;
00390 }
```

References [sClient::MarkForDelete](#).

Referenced by [DeleteClientByAccountNumber\(\)](#).

Here is the caller graph for this function:



4.1.2.15 PerfromMainMenueOption()

```
void PerfromMainMenueOption (
    enMainMenueOptions MainMenueOption )
```

メインメニューオプションを実行する関数

Parameters

<i>MainMenuOption</i>	(enMainMenuOptions) メインメニューのオプションを表す列挙型
-----------------------	---

この関数は、メインメニューのオプションを引数として受け取り、それに対応する操作を実行します。具体的な操作は、引数で指定されたオプションに依存します。

Definition at line 869 of file [Bank_System.cpp](#).

```

00870 {
00871     switch (MainMenuOption)
00872     {
00873     case enMainMenuOptions::eListClients:
00874     {
00875         system("cls");
00876         ShowAllClientsScreen();
00877         GoBackToMainMenu();
00878         break;
00879     }
00880     case enMainMenuOptions::eAddNewClient:
00881     {
00882         system("cls");
00883         ShowAddNewClientsScreen();
00884         GoBackToMainMenu();
00885         break;
00886     }
00887     case enMainMenuOptions::eDeleteClient:
00888     {
00889         system("cls");
00890         ShowDeleteClientScreen();
00891         GoBackToMainMenu();
00892         break;
00893     }
00894     case enMainMenuOptions::eUpdateClient:
00895     {
00896         system("cls");
00897         ShowUpdateClientScreen();
00898         GoBackToMainMenu();
00899         break;
00900     }
00901     case enMainMenuOptions::eFindClient:
00902     {
00903         system("cls");
00904         ShowFindClientScreen();
00905         GoBackToMainMenu();
00906         break;
00907     }
00908     case enMainMenuOptions::eShowTransactionsMenu:
00909     {
00910         system("cls");
00911         ShowTransactionsMenu();
00912         break;
00913     }
00914     case enMainMenuOptions::eExit:
00915     {
00916         system("cls");
00917         ShowEndScreen();
00918         break;
00919     }
00920     }
00921 }
```

References [eAddNewClient](#), [eDeleteClient](#), [eExit](#), [eFindClient](#), [eListClients](#), [eShowTransactionsMenu](#), [eUpdateClient](#), [GoBackToMainMenu\(\)](#), [ShowAddNewClientsScreen\(\)](#), [ShowAllClientsScreen\(\)](#), [ShowDeleteClientScreen\(\)](#), [ShowEndScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowTransactionsMenu\(\)](#), and [ShowUpdateClientScreen\(\)](#).

Referenced by [ShowMainMenu\(\)](#).


```

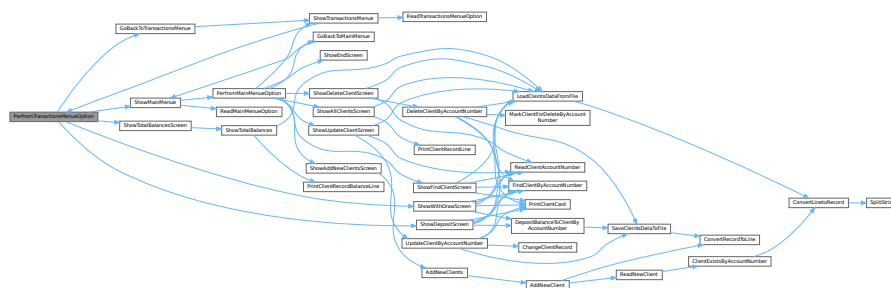
00807
00808 case enTransactionsMenuOptions::eWithdraw:
00809 {
00810     system("cls");
00811     ShowWithdrawScreen();
00812     GoBackToTransactionsMenu();
00813     break;
00814 }
00815
00816 case enTransactionsMenuOptions::eShowTotalBalance:
00817 {
00818     system("cls");
00819     ShowTotalBalancesScreen();
00820     GoBackToTransactionsMenu();
00821     break;
00822 }
00823
00824 case enTransactionsMenuOptions::eShowMainMenu:
00825 {
00826     ShowMainMenu();
00827 }
00828 }
00829 }
00830 }

```

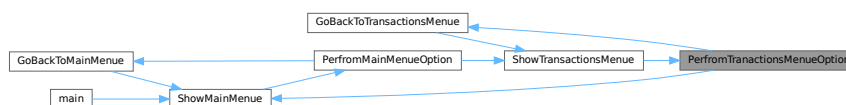
References [eDeposit](#), [eShowMainMenu](#), [eShowTotalBalance](#), [eWithdraw](#), [GoBackToTransactionsMenu\(\)](#), [ShowDepositScreen\(\)](#), [ShowMainMenu\(\)](#), [ShowTotalBalancesScreen\(\)](#), and [ShowWithdrawScreen\(\)](#).

Referenced by [ShowTransactionsMenu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.17 PrintClientCard()

```

void PrintClientCard (
    sClient Client )

```

クライアントのレコードを一行で出力する

Parameters

<i>Client</i>	(sClient) 出力するクライアントのレコード情報
---------------	-----------------------------

この関数は、引数で与えられたクライアントのレコード情報を一行で出力します。出力内容はクライアントの各フィールドがカンマ区切りの形式となります。出力は標準出力に送られます。

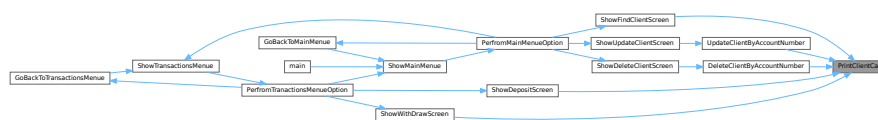
Definition at line 310 of file [Bank_System.cpp](#).

```
00311 {
00312     cout << "\nThe following are the client details:\n";
00313     cout << "-----\n";
00314     cout << "\nAccount Number: " << Client.AccountNumber;
00315     cout << "\nPin Code      : " << Client.PinCode;
00316     cout << "\nName         : " << Client.Name;
00317     cout << "\nPhone        : " << Client.Phone;
00318     cout << "\nAccount Balance: " << Client.AccountBalance;
00319     cout << "\n-----\n";
00320 }
```

References [sClient::AccountBalance](#), [sClient::AccountNumber](#), [sClient::Name](#), [sClient::Phone](#), and [sClient::PinCode](#).

Referenced by [DeleteClientByAccountNumber\(\)](#), [ShowDepositScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowWithdrawScreen\(\)](#), and [UpdateClientByAccountNumber\(\)](#).

Here is the caller graph for this function:



4.1.2.18 PrintClientRecordBalanceLine()

```
void PrintClientRecordBalanceLine (
    sClient Client )
```

クライアントのレコードを一行で出力する

Parameters

<i>Client</i>	(sClient) 出力するクライアントのレコード情報
---------------	-----------------------------

この関数は、引数で与えられたクライアントのレコード情報を一行で出力します。出力内容はクライアントの各フィールドがカンマ区切りの形式となります。出力は標準出力に送られます。

Definition at line 220 of file [Bank_System.cpp](#).

```
00221 {
00222
00223     cout << "|" << setw(15) << left << Client.AccountNumber;
00224     cout << "|" << setw(40) << left << Client.Name;
00225     cout << "|" << setw(12) << left << Client.AccountBalance;
00226 }
```

References [sClient::AccountBalance](#), [sClient::AccountNumber](#), and [sClient::Name](#).

Referenced by [ShowTotalBalances\(\)](#).

Here is the caller graph for this function:



4.1.2.19 PrintClientRecordLine()

```
void PrintClientRecordLine (
    sClient Client )
```

クライアントのレコードを一行で出力する

Parameters

<i>Client</i>	(sClient) 出力するクライアントのレコード情報
---------------	-----------------------------

この関数は、引数で与えられたクライアントのレコード情報を一行で出力します。出力内容はクライアントの各フィールドがカンマ区切りの形式となります。出力は標準出力に送られます。

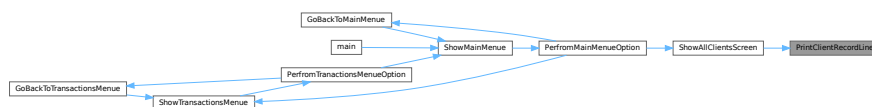
Definition at line 207 of file [Bank_System.cpp](#).

```
00208 {
00209
00210     cout << "|" << setw(15) << left << Client.AccountNumber;
00211     cout << "|" << setw(10) << left << Client.PinCode;
00212     cout << "|" << setw(40) << left << Client.Name;
00213     cout << "|" << setw(12) << left << Client.Phone;
00214     cout << "|" << setw(12) << left << Client.AccountBalance;
00215 }
```

References [sClient::AccountBalance](#), [sClient::AccountNumber](#), [sClient::Name](#), [sClient::Phone](#), and [sClient::PinCode](#).

Referenced by [ShowAllClientsScreen\(\)](#).

Here is the caller graph for this function:



4.1.2.20 ReadClientAccountNumber()

```
string ReadClientAccountNumber ( )
```

クライアントのアカウント番号を読み取る

Returns

string クライアントのアカウント番号を文字列として返す

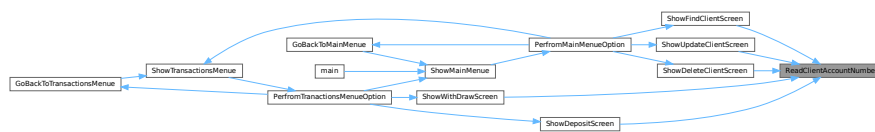
この関数は、クライアントのアカウント番号を読み取り、それを文字列として返します。具体的な読み取り方法やアカウント番号の形式は、実装に依存します。

Definition at line 589 of file [Bank_System.cpp](#).

```
00590 {
00591     string AccountNumber = "";
00592
00593     cout << "\nPlease enter AccountNumber? ";
00594     cin >> AccountNumber;
00595     return AccountNumber;
00596 }
```

Referenced by [ShowDeleteClientScreen\(\)](#), [ShowDepositScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowUpdateClientScreen\(\)](#), and [ShowWithdrawScreen\(\)](#).

Here is the caller graph for this function:

**4.1.2.21 ReadMainMenuOption()**

```
short ReadMainMenuOption ( )
```

トランザクションメニューオプションを読み込む関数

Returns

short 選択されたメニューオプションを表す短い整数

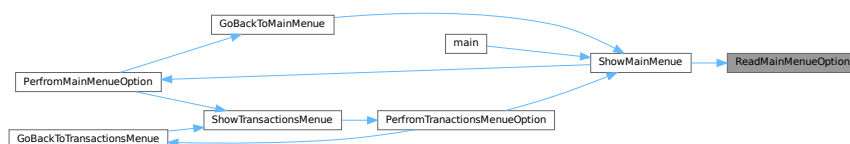
この関数は、ユーザーによるトランザクションメニューの選択を読み込み、選択されたメニューオプションを表す短い整数を返します。

Definition at line 853 of file [Bank_System.cpp](#).

```
00854 {
00855     short Choice = 0;
00856     do
00857     {
00858         cout << "Choose what do you want to do? [1 to 7]? ";
00859         cin >> Choice;
00860     } while (Choice < 1 || Choice > 7);
00861
00862     return Choice;
00863 }
```

Referenced by [ShowMainMenu\(\)](#).

Here is the caller graph for this function:



4.1.2.22 ReadNewClient()

`sClient` ReadNewClient ()

新規クライアントを読み込む

Returns

`sClient` 新規に読み込まれたクライアントの情報を保持する `sClient` オブジェクト

この関数は新規クライアントの情報を読み込み、その情報を保持する `sClient` オブジェクトを返します。

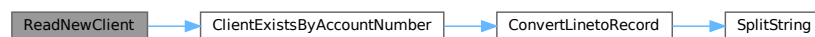
Definition at line 141 of file `Bank_System.cpp`.

```
00142 {
00143     sClient Client;
00144
00145     cout << "Enter Account Number? ";
00146
00147     // Usage of std::ws will extract all the whitespace character
00148     getline(cin >> ws, Client.AccountNumber);
00149
00150     while (ClientExistsByAccountNumber(Client.AccountNumber, ClientsFileName))
00151     {
00152         cout << "\nClient with [" << Client.AccountNumber << "] already exists, Enter another Account Number? ";
00153         getline(cin >> ws, Client.AccountNumber);
00154     }
00155
00156     cout << "Enter PinCode? ";
00157     getline(cin, Client.PinCode);
00158
00159     cout << "Enter Name? ";
00160     getline(cin, Client.Name);
00161
00162     cout << "Enter Phone? ";
00163     getline(cin, Client.Phone);
00164
00165     cout << "Enter AccountBalance? ";
00166     cin >> Client.AccountBalance;
00167
00168     return Client;
00169 }
```

References `sClient::AccountBalance`, `sClient::AccountNumber`, `ClientExistsByAccountNumber()`, `ClientsFileName`, `sClient::Name`, `sClient::Phone`, and `sClient::PinCode`.

Referenced by `AddNewClient()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.23 ReadTransactionsMenuOption()

```
short ReadTransactionsMenuOption ( )
```

トランザクションメニューオプションを読み込む関数

Returns

short 選択されたメニューオプションを表す短い整数

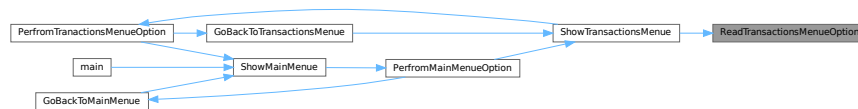
この関数は、ユーザーによるトランザクションメニューの選択を読み込み、選択されたメニューオプションを表す短い整数を返します。

Definition at line 783 of file [Bank_System.cpp](#).

```
00784 {
00785     cout << "Choose what do you want to do? [1 to 4]? ";
00786     short Choice = 0;
00787     cin >> Choice;
00788
00789     return Choice;
00790 }
```

Referenced by [ShowTransactionsMenu\(\)](#).

Here is the caller graph for this function:



4.1.2.24 SaveCleintsDataToFile()

```
vector< sClient > SaveCleintsDataToFile (
    string FileName,
    vector< sClient > vClients )
```

クライアントデータを指定のファイルに保存する関数

Parameters

<i>FileName</i>	(string) クライアントデータを保存するためのファイル名
<i>vClients</i>	(vector< sClient >) 保存するクライアントデータのベクター

Returns

vector< sClient > 保存に成功したクライアントデータのベクターを返す

この関数は、指定されたファイル名でクライアントのデータを保存します。保存に成功したクライアントのデータのみをベクターとして返します。保存に失敗した場合の処理はこの関数の中で行われます。

Definition at line 398 of file [Bank_System.cpp](#).

```
00399 {
00400
00401     fstream MyFile;
00402     MyFile.open(fileName, ios::out); // overwrite
00403
00404     string DataLine;
00405
00406     if (MyFile.is_open())
00407     {
00408         for (sClient C : vClients)
00409         {
00410             if (C.MarkForDelete == false)
00411             {
00412                 // we only write records that are not marked for delete.
00413                 DataLine = ConvertRecordToLine(C);
00414                 MyFile << DataLine << endl;
00415             }
00416         }
00417     }
00418     MyFile.close();
00419 }
00420
00421 return vClients;
00422 }
00423
00424 }
```

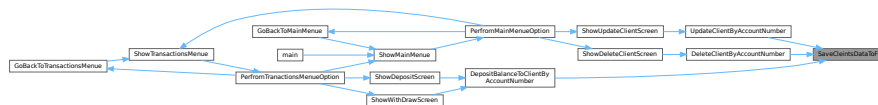
References [ConvertRecordToLine\(\)](#).

Referenced by [DeleteClientByAccountNumber\(\)](#), [DepositBalanceToClientByAccountNumber\(\)](#), and [UpdateClientByAccountNumber\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.25 ShowAddNewClientsScreen()

```
void ShowAddNewClientsScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 629 of file [Bank_System.cpp](#).

```

00630 {
00631     cout << "\n-----\n";
00632     cout << "\tAdd New Clients Screen";
00633     cout << "\n-----\n";
00634
00635     AddNewClients();
00636 }

```

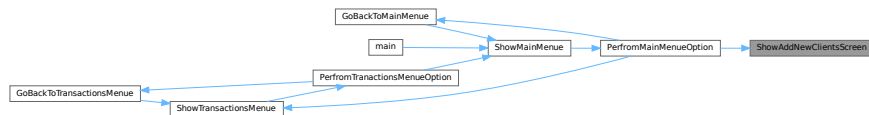
References [AddNewClients\(\)](#).

Referenced by [PerfromMainMenueOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.26 ShowAllClientsScreen()

```
void ShowAllClientsScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 231 of file [Bank_System.cpp](#).

```

00232 {
00233
00234     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00235
00236     cout << "\n\t\t\t\t\tClient List (" << vClients.size() << ") Client(s).";
00237     cout << "\n_____";
00238     cout << "\n";
00239     << endl;
00240
00241     cout << "| " << left << setw(15) << "Accout Number";
00242     cout << "| " << left << setw(10) << "Pin Code";
00243     cout << "| " << left << setw(40) << "Client Name";
00244     cout << "| " << left << setw(12) << "Phone";
00245     cout << "| " << left << setw(12) << "Balance";
00246     cout << "\n_____";
00247     cout << "\n";
00248     << endl;
00249
00250     if (vClients.size() == 0)
00251         cout << "\t\t\t\t\tNo Clients Available In the System!";
00252     else
00253     {
00254         for (sClient Client : vClients)
00255         {
00256

```

```

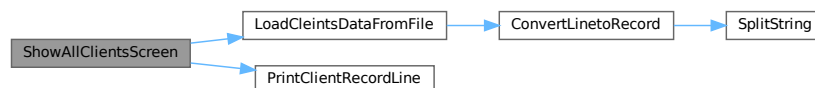
00257     PrintClientRecordLine (Client);
00258     cout << endl;
00259 }
00260
00261 cout << "\n_____";
00262 cout << "_____ \n"
00263     << endl;
00264 }

```

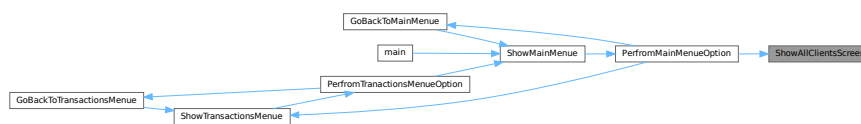
References [ClientsFileName](#), [LoadCleintsDataFromFile\(\)](#), and [PrintClientRecordLine\(\)](#).

Referenced by [PerfromMainMenueOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.27 ShowDeleteClientScreen()

```
void ShowDeleteClientScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 601 of file [Bank_System.cpp](#).

```

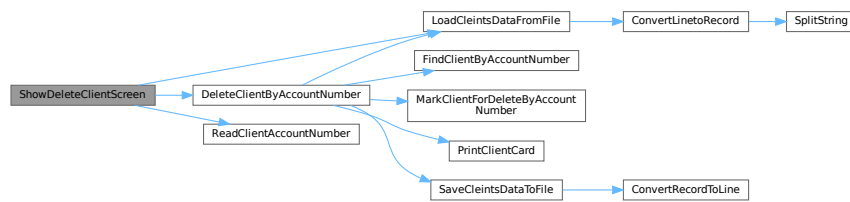
00602 {
00603     cout << "\n-----\n";
00604     cout << "\tDelete Client Screen";
00605     cout << "\n-----\n";
00606
00607     vector<sClient> vClients = LoadCleintsDataFromFile (ClientsFileName);
00608     string AccountNumber = ReadClientAccountNumber ();
00609     DeleteClientByAccountNumber (AccountNumber, vClients);
00610 }

```

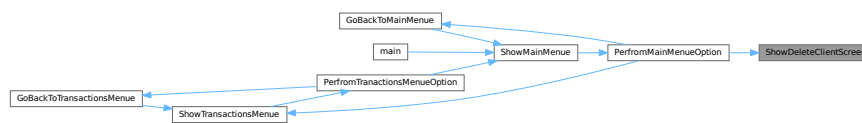
References [ClientsFileName](#), [DeleteClientByAccountNumber\(\)](#), [LoadCleintsDataFromFile\(\)](#), and [ReadClientAccountNumber\(\)](#).

Referenced by [PerfromMainMenueOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.28 ShowDepositScreen()

```
void ShowDepositScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 670 of file [Bank_System.cpp](#).

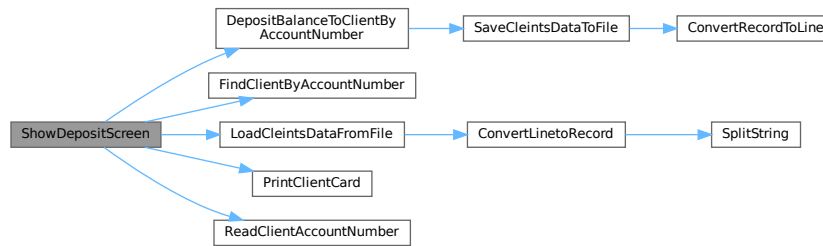
```

00671 {
00672     cout << "\n-----\n";
00673     cout << "\tDeposit Screen";
00674     cout << "\n-----\n";
00675
00676     sClient Client;
00677
00678     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00679     string AccountNumber = ReadClientAccountNumber();
00680
00681     while (!FindClientByAccountNumber(AccountNumber, vClients, Client))
00682     {
00683         cout << "\nClient with [" << AccountNumber << "] does not exist.\n";
00684         AccountNumber = ReadClientAccountNumber();
00685     }
00686
00687     PrintClientCard(Client);
00688
00689     double Amount = 0;
00690     cout << "\nPlease enter deposit amount? ";
00691     cin >> Amount;
00692
00693     DepositBalanceToClientByAccountNumber(AccountNumber, Amount, vClients);
00694 }
  
```

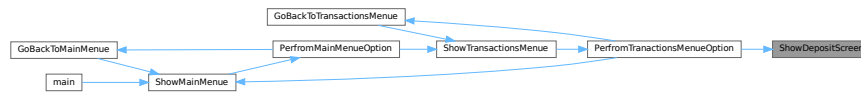
References [ClientsFileName](#), [DepositBalanceToClientByAccountNumber\(\)](#), [FindClientByAccountNumber\(\)](#), [LoadCleintsDataFromFile\(\)](#), [PrintClientCard\(\)](#), and [ReadClientAccountNumber\(\)](#).

Referenced by [PerfromTranactionsMenuueOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.29 ShowEndScreen()

```
void ShowEndScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

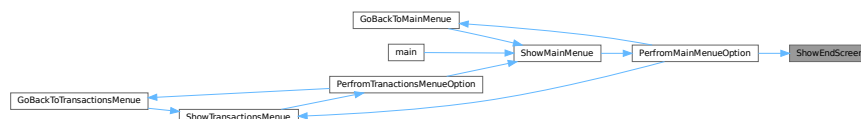
Definition at line 659 of file [Bank_System.cpp](#).

```

00660 {
00661     cout << "\n-----\n";
00662     cout << "\tProgram Ends :-)";
00663     cout << "\n-----\n";
00664     cout << " - Author: Oussama Azzouz";
00665 }
  
```

Referenced by [PerfromMainMenueOption\(\)](#).

Here is the caller graph for this function:



4.1.2.30 ShowFindClientScreen()

```
void ShowFindClientScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

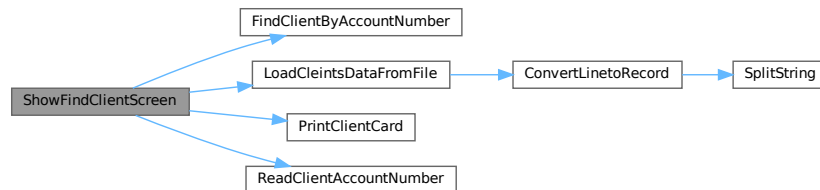
Definition at line 641 of file [Bank_System.cpp](#).

```
00642 {
00643     cout << "\n-----\n";
00644     cout << "\tFind Client Screen";
00645     cout << "\n-----\n";
00646
00647     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00648     sClient Client;
00649     string AccountNumber = ReadClientAccountNumber();
00650     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00651         PrintClientCard(Client);
00652     else
00653         cout << "\nClient with Account Number[" << AccountNumber << "] is not found!";
00654 }
```

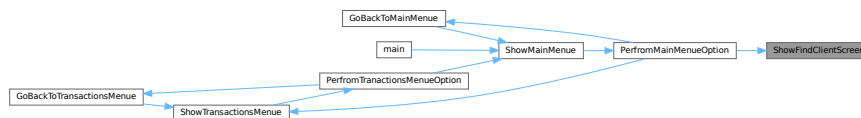
References [ClientsFileName](#), [FindClientByAccountNumber\(\)](#), [LoadCleintsDataFromFile\(\)](#), [PrintClientCard\(\)](#), and [ReadClientAccountNumber\(\)](#).

Referenced by [PerfromMainMenuOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.32 ShowTotalBalances()

```
void ShowTotalBalances ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

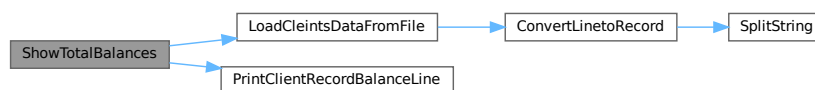
Definition at line 269 of file [Bank_System.cpp](#).

```
00270 {
00271
00272     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00273
00274     cout << "\n\t\t\t\t\tBalances List (" << vClients.size() << ") Client(s).";
00275     cout << "\n";
00276     cout << "_____\n";
00277     << endl;
00278
00279     cout << "| " << left << setw(15) << "Accout Number";
00280     cout << "| " << left << setw(40) << "Client Name";
00281     cout << "| " << left << setw(12) << "Balance";
00282     cout << "\n";
00283     cout << "_____\n";
00284     << endl;
00285
00286     double TotalBalances = 0;
00287
00288     if (vClients.size() == 0)
00289         cout << "\t\t\t\t\tNo Clients Available In the System!";
00290     else
00291     {
00292         for (sClient Client : vClients)
00293         {
00294             PrintClientRecordBalanceLine(Client);
00295             TotalBalances += Client.AccountBalance;
00296
00297             cout << endl;
00298         }
00299     }
00300
00301     cout << "\n";
00302     cout << "_____\n";
00303     << endl;
00304     cout << "\t\t\t\t\tTotal Balances = " << TotalBalances;
00305 }
```

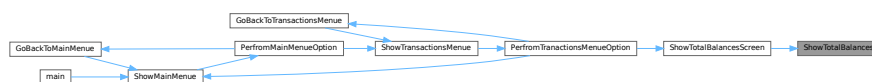
References [ClientsFileName](#), [LoadCleintsDataFromFile\(\)](#), and [PrintClientRecordBalanceLine\(\)](#).

Referenced by [ShowTotalBalancesScreen\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.33 ShowTotalBalancesScreen()

```
void ShowTotalBalancesScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

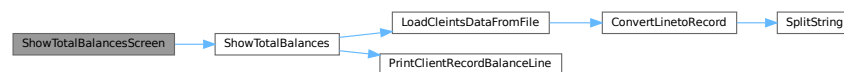
Definition at line 736 of file [Bank_System.cpp](#).

```
00737 {
00738
00739     ShowTotalBalances ();
00740 }
```

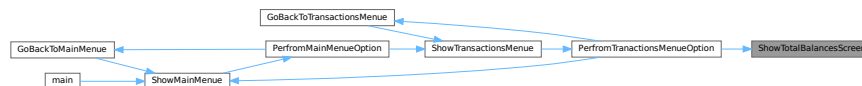
References [ShowTotalBalances\(\)](#).

Referenced by [PerfromTranactionsMenueOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.34 ShowTransactionsMenue()

```
void ShowTransactionsMenue ( )
```

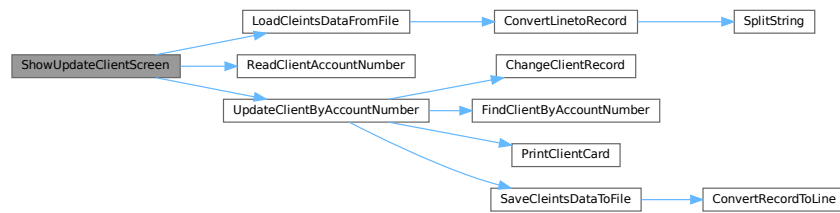
すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

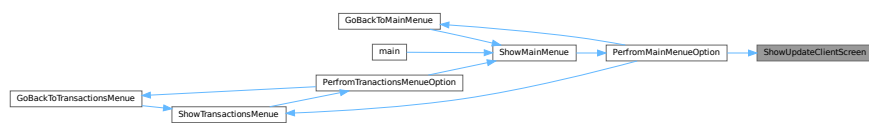
Definition at line 835 of file [Bank_System.cpp](#).

```
00836 {
00837     system("cls");
00838     cout << "=====\n";
00839     cout << "\t\tTransactions Menue Screen\n";
00840     cout << "=====\n";
00841     cout << "\t[1] Deposit.\n";
00842     cout << "\t[2] Withdraw.\n";
00843     cout << "\t[3] Total Balances.\n";
00844     cout << "\t[4] Main Menue.\n";
00845     cout << "=====\n";
00846     PerfromTranactionsMenueOption((enTransactionsMenueOptions) ReadTransactionsMenueOption());
```


Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.36 ShowWithdrawScreen()

```
void ShowWithdrawScreen ( )
```

すべてのクライアントの画面を表示する

この関数は、現在接続されているすべてのクライアントの画面を表示します。各クライアントの画面は、それぞれ独立して表示されます。この関数を呼び出すと、すべてのクライアントの画面が一覧できます。具体的な表示方法やレイアウトは、実装に依存します。

Definition at line 699 of file [Bank_System.cpp](#).

```

00700 {
00701     cout << "\n-----\n";
00702     cout << "\tWithdraw Screen";
00703     cout << "\n-----\n";
00704
00705     sClient Client;
00706
00707     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00708     string AccountNumber = ReadClientAccountNumber();
00709
00710     while (!FindClientByAccountNumber(AccountNumber, vClients, Client))
00711     {
00712         cout << "\nClient with [" << AccountNumber << "] does not exist.\n";
00713         AccountNumber = ReadClientAccountNumber();
00714     }
00715
00716     PrintClientCard(Client);
00717
00718     double Amount = 0;
00719     cout << "\nPlease enter withdraw amount? ";
00720     cin >> Amount;
00721
00722     // Validate that the amount does not exceeds the balance
00723     while (Amount > Client.AccountBalance)
00724     {
00725         cout << "\nAmount Exceeds the balance, you can withdraw up to : " << Client.AccountBalance << endl;
00726         cout << "Please enter another amount? ";
00727         cin >> Amount;
00728     }
00729
00730     DepositBalanceToClientByAccountNumber(AccountNumber, Amount * -1, vClients);

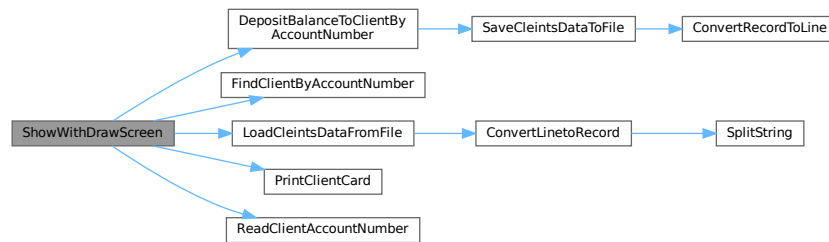
```

```
00731 }
```

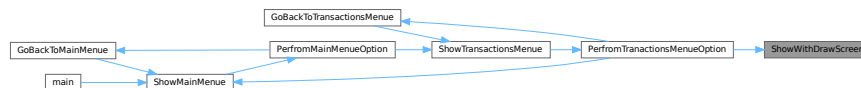
References [sClient::AccountBalance](#), [ClientsFileName](#), [DepositBalanceToClientByAccountNumber\(\)](#), [FindClientByAccountNumber\(\)](#), [LoadCleintsDataFromFile\(\)](#), [PrintClientCard\(\)](#), and [ReadClientAccountNumber\(\)](#).

Referenced by [PerfromTranactionsMenuOption\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.37 SplitString()

```
vector< string > SplitString (
    string S1,
    string Delim )
```

文字列を指定した区切り文字で分割する

Parameters

<i>S1</i>	(string) 分割対象の文字列
<i>Delim</i>	(string) 区切り文字

Returns

`vector< string >` 分割された文字列のリスト

入力された文字列S1 をDelim で指定された区切り文字で分割し、その結果を文字列のベクターとして返します。Delim が空文字列の場合、S1 は一文字ずつ分割されます。

Definition at line 29 of file [Bank_System.cpp](#).


```

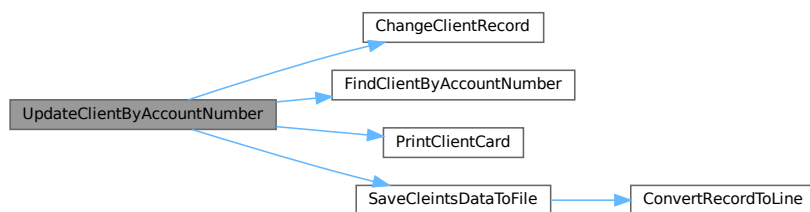
00520     sClient Client;
00521     char Answer = 'n';
00522
00523     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00524     {
00525
00526         PrintClientCard(Client);
00527         cout << "\n\nAre you sure you want update this client? y/n ? ";
00528         cin >> Answer;
00529         if (Answer == 'y' || Answer == 'Y')
00530         {
00531
00532             for (sClient &C : vClients)
00533             {
00534                 if (C.AccountNumber == AccountNumber)
00535                 {
00536                     C = ChangeClientRecord(AccountNumber);
00537                     break;
00538                 }
00539             }
00540
00541             SaveCleintsDataToFile(ClientsFileName, vClients);
00542
00543             cout << "\n\nClient Updated Successfully.";
00544             return true;
00545         }
00546     }
00547     else
00548     {
00549         cout << "\n\nClient with Account Number (" << AccountNumber << ") is Not Found!";
00550         return false;
00551     }
00552 }

```

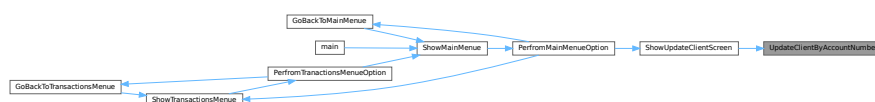
References [ChangeClientRecord\(\)](#), [ClientsFileName](#), [FindClientByAccountNumber\(\)](#), [PrintClientCard\(\)](#), and [SaveCleintsDataToFile\(\)](#).

Referenced by [ShowUpdateClientScreen\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.3 Variable Documentation

4.1.3.1 ClientsFileName

```
const string ClientsFileName = "Clients.txt"
```

Definition at line 8 of file [Bank_System.cpp](#).

Referenced by [AddNewClient\(\)](#), [DeleteClientByAccountNumber\(\)](#), [DepositBalanceToClientByAccountNumber\(\)](#), [ReadNewClient\(\)](#), [ShowAllClientsScreen\(\)](#), [ShowDeleteClientScreen\(\)](#), [ShowDepositScreen\(\)](#), [ShowFindClientScreen\(\)](#), [ShowTotalBalances\(\)](#), [ShowUpdateClientScreen\(\)](#), [ShowWithdrawScreen\(\)](#), and [UpdateClientByAccountNumber\(\)](#).

4.2 Bank_System.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include <fstream>
00003 #include <string>
00004 #include <vector>
00005 #include <iomanip>
00006
00007 using namespace std;
00008 const string ClientsFileName = "Clients.txt";
00009
00010 void ShowMainMenue();
00011 void ShowTransactionsMenue();
00012
00013 struct sClient
00014 {
00015     string AccountNumber;
00016     string PinCode;
00017     string Name;
00018     string Phone;
00019     double AccountBalance;
00020     bool MarkForDelete = false;
00021 };
00022
00029 vector<string> SplitString(string S1, string Delim)
00030 {
00031     vector<string> vString;
00032
00033     short pos = 0;
00034     string sWord; // define a string variable
00035
00036     // use find() function to get the position of the delimiters
00037     while ((pos = S1.find(Delim)) != std::string::npos)
00038     {
00039         sWord = S1.substr(0, pos); // store the word
00040         if (sWord != "")
00041         {
00042             vString.push_back(sWord);
00043         }
00044         S1.erase(0, pos + Delim.length()); /* erase() until positon and move to next word. */
00045     }
00046
00047     if (S1 != "")
00048     {
00049         vString.push_back(S1); // it adds last word of the string.
00050     }
00051
00052     return vString;
00053 }
00054
00062 sClient ConvertLinetoRecord(string Line, string Seperator = "#//")
00063 {
00064     sClient Client;
00065     vector<string> vClientData;
00066
00067     vClientData = SplitString(Line, Seperator);
00068
00069     Client.AccountNumber = vClientData[0];
00070     Client.PinCode = vClientData[1];
00071     Client.Name = vClientData[2];
00072     Client.Phone = vClientData[3];
00073     Client.AccountBalance = stod(vClientData[4]); // cast string to double
00074
00075     return Client;
00076 }
00077
00084 string ConvertRecordToLine(sClient Client, string Seperator = "#//")
00085 {
00086     string stClientRecord = "";
00087
00088

```

```

00089     stClientRecord += Client.AccountNumber + Seperator;
00090     stClientRecord += Client.PinCode + Seperator;
00091     stClientRecord += Client.Name + Seperator;
00092     stClientRecord += Client.Phone + Seperator;
00093     stClientRecord += to_string(Client.AccountBalance);
00094
00095     return stClientRecord;
00096 }
00097
00104 bool ClientExistsByAccountNumber(string AccountNumber, string FileName)
00105 {
00106
00107     vector<sClient> vClients;
00108
00109     fstream MyFile;
00110     MyFile.open(FileName, ios::in); // read Mode
00111
00112     if (MyFile.is_open())
00113     {
00114
00115         string Line;
00116         sClient Client;
00117
00118         while (getline(MyFile, Line))
00119         {
00120
00121             Client = ConvertLinetoRecord(Line);
00122             if (Client.AccountNumber == AccountNumber)
00123             {
00124                 MyFile.close();
00125                 return true;
00126             }
00127
00128             vClients.push_back(Client);
00129         }
00130
00131         MyFile.close();
00132     }
00133
00134     return false;
00135 }
00136
00141 sClient ReadNewClient()
00142 {
00143     sClient Client;
00144
00145     cout << "Enter Account Number? ";
00146
00147     // Usage of std::ws will extract allthe whitespace character
00148     getline(cin >> ws, Client.AccountNumber);
00149
00150     while (ClientExistsByAccountNumber(Client.AccountNumber, ClientsFileName))
00151     {
00152         cout << "\nClient with [" << Client.AccountNumber << "] already exists, Enter another Account Number?";
00153         getline(cin >> ws, Client.AccountNumber);
00154     }
00155
00156     cout << "Enter PinCode? ";
00157     getline(cin, Client.PinCode);
00158
00159     cout << "Enter Name? ";
00160     getline(cin, Client.Name);
00161
00162     cout << "Enter Phone? ";
00163     getline(cin, Client.Phone);
00164
00165     cout << "Enter AccountBalance? ";
00166     cin >> Client.AccountBalance;
00167
00168     return Client;
00169 }
00170
00176 vector<sClient> LoadCleintsDataFromFile(string FileName)
00177 {
00178
00179     vector<sClient> vClients;
00180
00181     fstream MyFile;
00182     MyFile.open(FileName, ios::in); // read Mode
00183
00184     if (MyFile.is_open())
00185     {
00186
00187         string Line;
00188         sClient Client;
00189

```

```

00190     while (getline(MyFile, Line))
00191     {
00192
00193         Client = ConvertLinetoRecord(Line);
00194
00195         vClients.push_back(Client);
00196     }
00197
00198     MyFile.close();
00199 }
00200
00201 return vClients;
00202 }
00203
00207 void PrintClientRecordLine(sClient Client)
00208 {
00209
00210     cout << "| " << setw(15) << left << Client.AccountNumber;
00211     cout << "| " << setw(10) << left << Client.PinCode;
00212     cout << "| " << setw(40) << left << Client.Name;
00213     cout << "| " << setw(12) << left << Client.Phone;
00214     cout << "| " << setw(12) << left << Client.AccountBalance;
00215 }
00216
00220 void PrintClientRecordBalanceLine(sClient Client)
00221 {
00222
00223     cout << "| " << setw(15) << left << Client.AccountNumber;
00224     cout << "| " << setw(40) << left << Client.Name;
00225     cout << "| " << setw(12) << left << Client.AccountBalance;
00226 }
00227
00231 void ShowAllClientsScreen()
00232 {
00233
00234     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00235
00236     cout << "\n\t\t\t\t\tClient List (" << vClients.size() << ") Client(s).";
00237     cout << "\n_____";
00238     cout << "\n";
00239     << endl;
00240
00241     cout << "| " << left << setw(15) << "Accout Number";
00242     cout << "| " << left << setw(10) << "Pin Code";
00243     cout << "| " << left << setw(40) << "Client Name";
00244     cout << "| " << left << setw(12) << "Phone";
00245     cout << "| " << left << setw(12) << "Balance";
00246     cout << "\n_____";
00247     cout << "\n";
00248     << endl;
00249
00250     if (vClients.size() == 0)
00251         cout << "\t\t\t\t\tNo Clients Available In the System!";
00252     else
00253     {
00254         for (sClient Client : vClients)
00255         {
00256
00257             PrintClientRecordLine(Client);
00258             cout << endl;
00259         }
00260
00261         cout << "\n_____";
00262         cout << "\n";
00263         << endl;
00264     }
00265
00269 void ShowTotalBalances()
00270 {
00271
00272     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00273
00274     cout << "\n\t\t\t\t\tBalances List (" << vClients.size() << ") Client(s).";
00275     cout << "\n_____";
00276     cout << "\n";
00277     << endl;
00278
00279     cout << "| " << left << setw(15) << "Accout Number";
00280     cout << "| " << left << setw(40) << "Client Name";
00281     cout << "| " << left << setw(12) << "Balance";
00282     cout << "\n_____";
00283     cout << "\n";
00284     << endl;
00285
00286     double TotalBalances = 0;
00287
00288     if (vClients.size() == 0)

```

```

00289     cout << "\t\t\t\t\tNo Clients Available In the System!";
00290 else
00291
00292     for (sClient Client : vClients)
00293     {
00294
00295         PrintClientRecordBalanceLine(Client);
00296         TotalBalances += Client.AccountBalance;
00297
00298         cout << endl;
00299     }
00300
00301     cout << "\n_____";
00302     cout << "_____\\n"
00303         << endl;
00304     cout << "\t\t\t\t\tTotal Balances = " << TotalBalances;
00305 }
00306
00310 void PrintClientCard(sClient Client)
00311 {
00312     cout << "\nThe following are the client details:\n";
00313     cout << "-----";
00314     cout << "\nAccount Number: " << Client.AccountNumber;
00315     cout << "\nPin Code       : " << Client.PinCode;
00316     cout << "\nName           : " << Client.Name;
00317     cout << "\nPhone          : " << Client.Phone;
00318     cout << "\nAccount Balance: " << Client.AccountBalance;
00319     cout << "\n-----\\n";
00320 }
00321
00329 bool FindClientByAccountNumber(string AccountNumber, vector<sClient> vClients, sClient &Client)
00330 {
00331
00332     for (sClient C : vClients)
00333     {
00334
00335         if (C.AccountNumber == AccountNumber)
00336         {
00337             Client = C;
00338             return true;
00339         }
00340     }
00341     return false;
00342 }
00343
00349 sClient ChangeClientRecord(string AccountNumber)
00350 {
00351     sClient Client;
00352
00353     Client.AccountNumber = AccountNumber;
00354
00355     cout << "\n\nEnter PinCode? ";
00356     getline(cin >> ws, Client.PinCode);
00357
00358     cout << "Enter Name? ";
00359     getline(cin, Client.Name);
00360
00361     cout << "Enter Phone? ";
00362     getline(cin, Client.Phone);
00363
00364     cout << "Enter AccountBalance? ";
00365     cin >> Client.AccountBalance;
00366
00367     return Client;
00368 }
00369
00376 bool MarkClientForDeleteByAccountNumber(string AccountNumber, vector<sClient> &vClients)
00377 {
00378
00379     for (sClient &C : vClients)
00380     {
00381
00382         if (C.AccountNumber == AccountNumber)
00383         {
00384             C.MarkForDelete = true;
00385             return true;
00386         }
00387     }
00388
00389     return false;
00390 }
00391
00398 vector<sClient> SaveCleintsDataToFile(string FileName, vector<sClient> vClients)
00399 {
00400
00401     ofstream MyFile;
00402     MyFile.open(FileName, ios::out); // overwrite

```

```

00403
00404     string DataLine;
00405
00406     if (MyFile.is_open())
00407     {
00408
00409         for (sClient C : vClients)
00410         {
00411
00412             if (C.MarkForDelete == false)
00413             {
00414                 // we only write records that are not marked for delete.
00415                 DataLine = ConvertRecordToLine(C);
00416                 MyFile << DataLine << endl;
00417             }
00418         }
00419
00420         MyFile.close();
00421     }
00422
00423     return vClients;
00424 }
00425
00447 void AddNewClient()
00448 {
00449     sClient Client;
00450     Client = ReadNewClient();
00451     AddDataLineToFile(ClientsFileName, ConvertRecordToLine(Client));
00452 }
00453
00457 void AddNewClients()
00458 {
00459     char AddMore = 'Y';
00460     do
00461     {
00462         // system("cls");
00463         cout << "Adding New Client:\n\n";
00464
00465         AddNewClient();
00466         cout << "\nClient Added Successfully, do you want to add more clients? Y/N? ";
00467
00468         cin >> AddMore;
00469
00470     } while (toupper(AddMore) == 'Y');
00471 }
00472
00479 bool DeleteClientByAccountNumber(string AccountNumber, vector<sClient> &vClients)
00480 {
00481
00482     sClient Client;
00483     char Answer = 'n';
00484
00485     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00486     {
00487
00488         PrintClientCard(Client);
00489
00490         cout << "\n\nAre you sure you want delete this client? y/n ? ";
00491         cin >> Answer;
00492         if (Answer == 'y' || Answer == 'Y')
00493         {
00494             MarkClientForDeleteByAccountNumber(AccountNumber, vClients);
00495             SaveCleintsDataToFile(ClientsFileName, vClients);
00496
00497             // Refresh Clients
00498             vClients = LoadCleintsDataFromFile(ClientsFileName);
00499
00500             cout << "\n\nClient Deleted Successfully.";
00501             return true;
00502         }
00503     }
00504     else
00505     {
00506         cout << "\nClient with Account Number (" << AccountNumber << ") is Not Found!";
00507         return false;
00508     }
00509 }
00510
00517 bool UpdateClientByAccountNumber(string AccountNumber, vector<sClient> &vClients)
00518 {
00519
00520     sClient Client;
00521     char Answer = 'n';
00522
00523     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00524     {
00525

```

```

00526     PrintClientCard(Client);
00527     cout << "\n\nAre you sure you want update this client? y/n ? ";
00528     cin >> Answer;
00529     if (Answer == 'y' || Answer == 'Y')
00530     {
00531
00532         for (sClient &C : vClients)
00533         {
00534             if (C.AccountNumber == AccountNumber)
00535             {
00536                 C = ChangeClientRecord(AccountNumber);
00537                 break;
00538             }
00539         }
00540
00541         SaveCleintsDataToFile(ClientsFileName, vClients);
00542
00543         cout << "\n\nClient Updated Successfully.";
00544         return true;
00545     }
00546 }
00547 else
00548 {
00549     cout << "\nClient with Account Number (" << AccountNumber << ") is Not Found!";
00550     return false;
00551 }
00552 }
00553
00561 bool DepositBalanceToClientByAccountNumber(string AccountNumber, double Amount, vector<sClient>
    &vClients)
00562 {
00563     char Answer = 'n';
00564
00565     cout << "\n\nAre you sure you want perfrom this transaction? y/n ? ";
00566     cin >> Answer;
00567     if (Answer == 'y' || Answer == 'Y')
00568     {
00569
00570         for (sClient &C : vClients)
00571         {
00572             if (C.AccountNumber == AccountNumber)
00573             {
00574                 C.AccountBalance += Amount;
00575                 SaveCleintsDataToFile(ClientsFileName, vClients);
00576                 cout << "\n\nDone Successfully. New balance is: " << C.AccountBalance;
00577
00578                 return true;
00579             }
00580         }
00581         return false;
00582     }
00583 }
00584
00589 string ReadClientAccountNumber()
00590 {
00591     string AccountNumber = "";
00592
00593     cout << "\nPlease enter AccountNumber? ";
00594     cin >> AccountNumber;
00595     return AccountNumber;
00596 }
00597
00601 void ShowDeleteClientScreen()
00602 {
00603     cout << "\n-----\n";
00604     cout << "\tDelete Client Screen";
00605     cout << "\n-----\n";
00606
00607     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00608     string AccountNumber = ReadClientAccountNumber();
00609     DeleteClientByAccountNumber(AccountNumber, vClients);
00610 }
00611
00615 void ShowUpdateClientScreen()
00616 {
00617     cout << "\n-----\n";
00618     cout << "\tUpdate Client Info Screen";
00619     cout << "\n-----\n";
00620
00621     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00622     string AccountNumber = ReadClientAccountNumber();
00623     UpdateClientByAccountNumber(AccountNumber, vClients);
00624 }
00625
00629 void ShowAddNewClientsScreen()
00630 {
00631     cout << "\n-----\n";

```

```

00632     cout << "\tAdd New Clients Screen";
00633     cout << "\n-----\n";
00634
00635     AddNewClients();
00636 }
00637
00641 void ShowFindClientScreen()
00642 {
00643     cout << "\n-----\n";
00644     cout << "\tFind Client Screen";
00645     cout << "\n-----\n";
00646
00647     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00648     sClient Client;
00649     string AccountNumber = ReadClientAccountNumber();
00650     if (FindClientByAccountNumber(AccountNumber, vClients, Client))
00651         PrintClientCard(Client);
00652     else
00653         cout << "\nClient with Account Number[" << AccountNumber << "] is not found!";
00654 }
00655
00659 void ShowEndScreen()
00660 {
00661     cout << "\n-----\n";
00662     cout << "\tProgram Ends :-)";
00663     cout << "\n-----\n";
00664     cout << " - Author: Oussama Azzouz";
00665 }
00666
00670 void ShowDepositScreen()
00671 {
00672     cout << "\n-----\n";
00673     cout << "\tDeposit Screen";
00674     cout << "\n-----\n";
00675
00676     sClient Client;
00677
00678     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00679     string AccountNumber = ReadClientAccountNumber();
00680
00681     while (!FindClientByAccountNumber(AccountNumber, vClients, Client))
00682     {
00683         cout << "\nClient with [" << AccountNumber << "] does not exist.\n";
00684         AccountNumber = ReadClientAccountNumber();
00685     }
00686
00687     PrintClientCard(Client);
00688
00689     double Amount = 0;
00690     cout << "\nPlease enter deposit amount? ";
00691     cin >> Amount;
00692
00693     DepositBalanceToClientByAccountNumber(AccountNumber, Amount, vClients);
00694 }
00695
00699 void ShowWithdrawScreen()
00700 {
00701     cout << "\n-----\n";
00702     cout << "\tWithdraw Screen";
00703     cout << "\n-----\n";
00704
00705     sClient Client;
00706
00707     vector<sClient> vClients = LoadCleintsDataFromFile(ClientsFileName);
00708     string AccountNumber = ReadClientAccountNumber();
00709
00710     while (!FindClientByAccountNumber(AccountNumber, vClients, Client))
00711     {
00712         cout << "\nClient with [" << AccountNumber << "] does not exist.\n";
00713         AccountNumber = ReadClientAccountNumber();
00714     }
00715
00716     PrintClientCard(Client);
00717
00718     double Amount = 0;
00719     cout << "\nPlease enter withdraw amount? ";
00720     cin >> Amount;
00721
00722     // Validate that the amount does not exceeds the balance
00723     while (Amount > Client.AccountBalance)
00724     {
00725         cout << "\nAmount Exceeds the balance, you can withdraw up to : " << Client.AccountBalance << endl;
00726         cout << "Please enter another amount? ";
00727         cin >> Amount;
00728     }
00729
00730     DepositBalanceToClientByAccountNumber(AccountNumber, Amount * -1, vClients);

```



```

00731 }
00732
00736 void ShowTotalBalancesScreen()
00737 {
00738     ShowTotalBalances();
00740 }
00741
00742 enum enTransactionsMenuOptions
00743 {
00744     eDeposit = 1,
00745     eWithdraw = 2,
00746     eShowTotalBalance = 3,
00747     eShowMainMenu = 4
00748 };
00749
00750 enum enMainMenuOptions
00751 {
00752     eListClients = 1,
00753     eAddNewClient = 2,
00754     eDeleteClient = 3,
00755     eUpdateClient = 4,
00756     eFindClient = 5,
00757     eShowTransactionsMenu = 6,
00758     eExit = 7
00759 };
00760
00764 void GoBackToMainMenu()
00765 {
00766     cout << "\n\nPress any key to go back to Main Menue...";
00767     system("pause>0");
00768     ShowMainMenu();
00769 }
00773 void GoBackToTransactionsMenu()
00774 {
00775     cout << "\n\nPress any key to go back to Transactions Menue...";
00776     system("pause>0");
00777     ShowTransactionsMenu();
00778 }
00783 short ReadTransactionsMenuOption()
00784 {
00785     cout << "Choose what do you want to do? [1 to 4]? ";
00786     short Choice = 0;
00787     cin >> Choice;
00788
00789     return Choice;
00790 }
00791
00796 void PerfromTranactionsMenuOption(enTransactionsMenuOptions TransactionMenuOption)
00797 {
00798     switch (TransactionMenuOption)
00799     {
00800     case enTransactionsMenuOptions::eDeposit:
00801     {
00802         system("cls");
00803         ShowDepositScreen();
00804         GoBackToTransactionsMenu();
00805         break;
00806     }
00807
00808     case enTransactionsMenuOptions::eWithdraw:
00809     {
00810         system("cls");
00811         ShowWithdrawScreen();
00812         GoBackToTransactionsMenu();
00813         break;
00814     }
00815
00816     case enTransactionsMenuOptions::eShowTotalBalance:
00817     {
00818         system("cls");
00819         ShowTotalBalancesScreen();
00820         GoBackToTransactionsMenu();
00821         break;
00822     }
00823
00824     case enTransactionsMenuOptions::eShowMainMenu:
00825     {
00826         ShowMainMenu();
00827     }
00828     }
00829 }
00830
00831
00835 void ShowTransactionsMenu()
00836 {
00837     system("cls");

```

```

00838     cout << "=====\n";
00839     cout << "\t\tTransactions Menu Screen\n";
00840     cout << "=====\n";
00841     cout << "\t[1] Deposit.\n";
00842     cout << "\t[2] Withdraw.\n";
00843     cout << "\t[3] Total Balances.\n";
00844     cout << "\t[4] Main Menu.\n";
00845     cout << "=====\n";
00846     PerfromTranactionsMenuOption((enTransactionsMenuOptions)ReadTransactionsMenuOption());
00847 }
00848
00853 short ReadMainMenuOption()
00854 {
00855     short Choice = 0;
00856     do
00857     {
00858         cout << "Choose what do you want to do? [1 to 7]? ";
00859         cin >> Choice;
00860     } while (Choice < 1 || Choice > 7);
00861
00862     return Choice;
00863 }
00864
00869 void PerfromMainMenuOption(enMainMenuOptions MainMenuOption)
00870 {
00871     switch (MainMenuOption)
00872     {
00873     case enMainMenuOptions::eListClients:
00874     {
00875         system("cls");
00876         ShowAllClientsScreen();
00877         GoBackToMainMenu();
00878         break;
00879     }
00880     case enMainMenuOptions::eAddNewClient:
00881     {
00882         system("cls");
00883         ShowAddNewClientsScreen();
00884         GoBackToMainMenu();
00885         break;
00886     }
00887     case enMainMenuOptions::eDeleteClient:
00888     {
00889         system("cls");
00890         ShowDeleteClientScreen();
00891         GoBackToMainMenu();
00892         break;
00893     }
00894     case enMainMenuOptions::eUpdateClient:
00895     {
00896         system("cls");
00897         ShowUpdateClientScreen();
00898         GoBackToMainMenu();
00899         break;
00900     }
00901     case enMainMenuOptions::eFindClient:
00902     {
00903         system("cls");
00904         ShowFindClientScreen();
00905         GoBackToMainMenu();
00906         break;
00907     }
00908     case enMainMenuOptions::eShowTransactionsMenu:
00909     {
00910         system("cls");
00911         ShowTransactionsMenu();
00912         break;
00913     }
00914     case enMainMenuOptions::eExit:
00915     {
00916         system("cls");
00917         ShowEndScreen();
00918         break;
00919     }
00920 }
00921
00919 void ShowMainMenu()
00920 {
00921     system("cls");
00922     cout << "=====\n";
00923     cout << "\t\tMain Menu Screen\n";
00924     cout << "=====\n";
00925     cout << "\t[1] Show Client List.\n";
00926     cout << "\t[2] Add New Client.\n";
00927     cout << "\t[3] Delete Client.\n";
00928     cout << "\t[4] Update Client Info.\n";
00929     cout << "\t[5] Find Client.\n";
00930     cout << "\t[6] Transactions.\n";
00931     cout << "\t[7] Exit.\n";
00932     cout << "=====\n";
00933     PerfromMainMenuOption((enMainMenuOptions)ReadMainMenuOption());
00934 }
00935

```

```
00940 int main()
00941 {
00942     ShowMainMenue();
00943     system("pause>0");
00944     return 0;
00945 }
```

